

Architecture

- of the Res Medicinae Medical Information System -

Copyright (c) 1999-2002. Christian Heller. All rights reserved.
Distributable under the GNU FDL License.
<http://www.resmedicinae.org> <christian.heller@tuxtax.de>



Contents



1 Basics

- 1.1 Information Processing
- 1.2 Analysis and Requirements
- 1.3 Logical Architecture (Layers)
- 1.4 Package Dependencies
- 1.5 Directory Structure
- 1.6 Physical Architecture (Tiers)

2 Application Components

- 2.1 Default Application
- 2.2 Separation of Concerns
- 2.3 Component Lifecycle
- 2.4 Inversion of Control

3 Domain Modelling

- 3.1 Single Model
- 3.2 Semi-Structured Model
- 3.3 Knowledge Representation
- 3.4 Interoperability (Dual Model)
- 3.5 Current Approach

4 Appendix

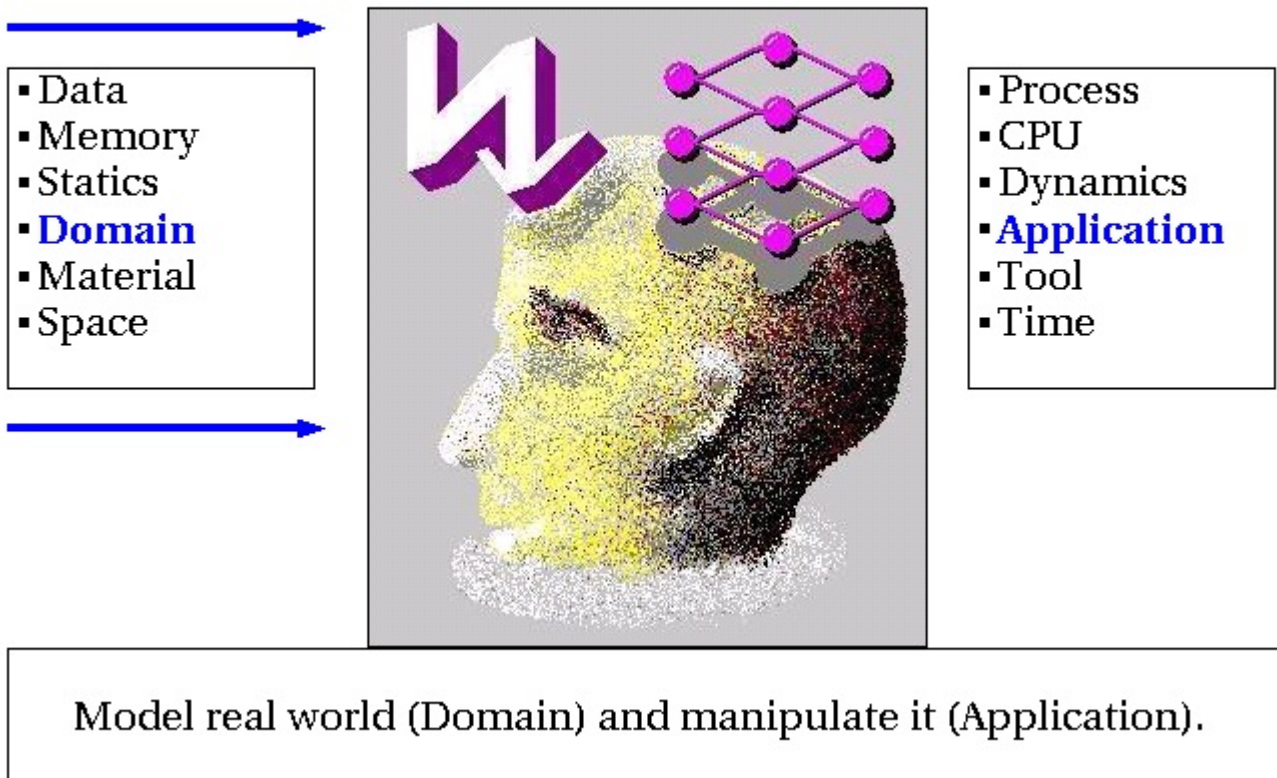
- 4.1 Support
- 4.2 Abbreviations
- 4.3 Sources
- 4.4 Tools

X Tutorial

- X.1 Build Process

1 Basics

1.1 Information Processing

- 
- The diagram illustrates the components of information processing. It features a central image of a human head in profile, composed of a point cloud. Above the head is a stylized 'W' logo. To the right of the head is a neural network diagram with three layers of nodes. Two blue arrows point from the left list towards the central image. Two blue arrows point from the right list towards the central image. The entire diagram is enclosed in a rectangular frame.
- Data
 - Memory
 - Statics
 - **Domain**
 - Material
 - Space

- Process
- CPU
- Dynamics
- **Application**
- Tool
- Time

Model real world (Domain) and manipulate it (Application).

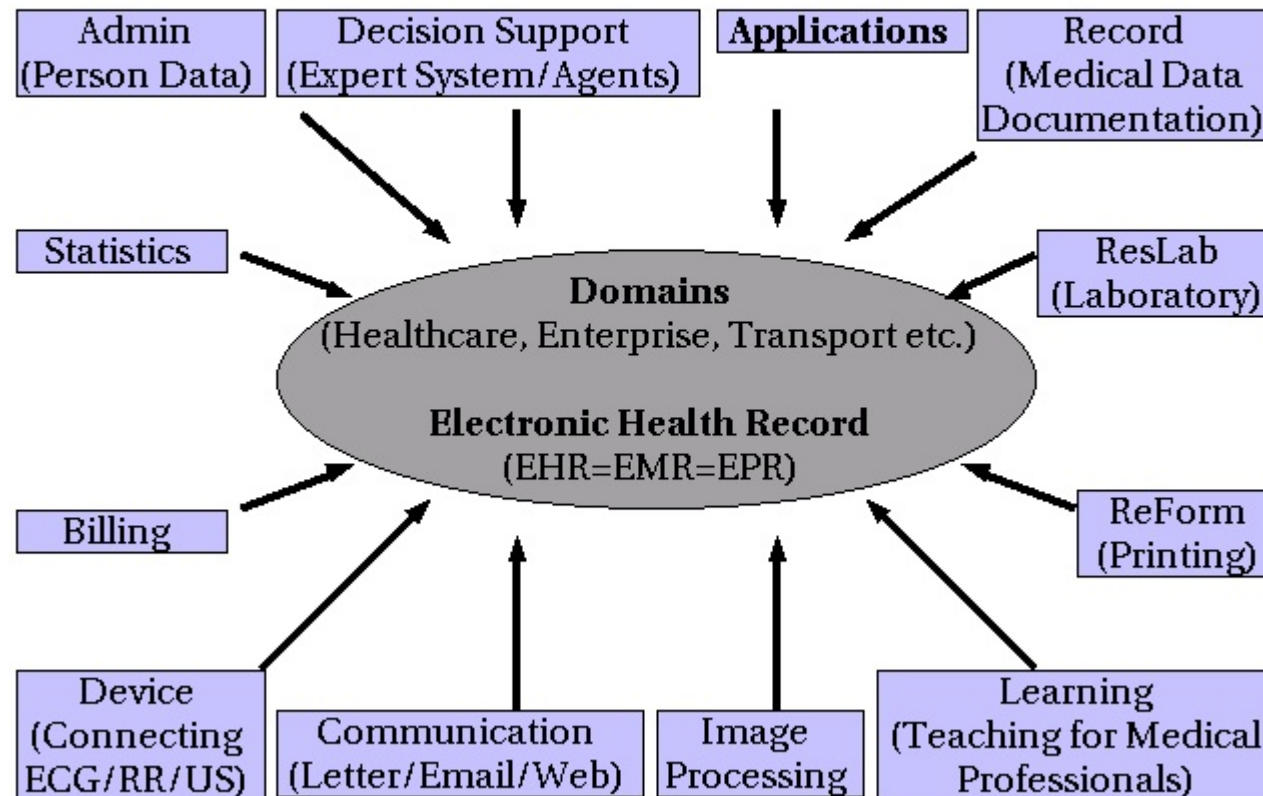
1.2 Analysis and Requirements



Analysis Document grows fast and is already quite complex due to heavy contribution of about 80 German Medical Doctors discussing in the resmedicinae-deutsch mailing list. Tasks:

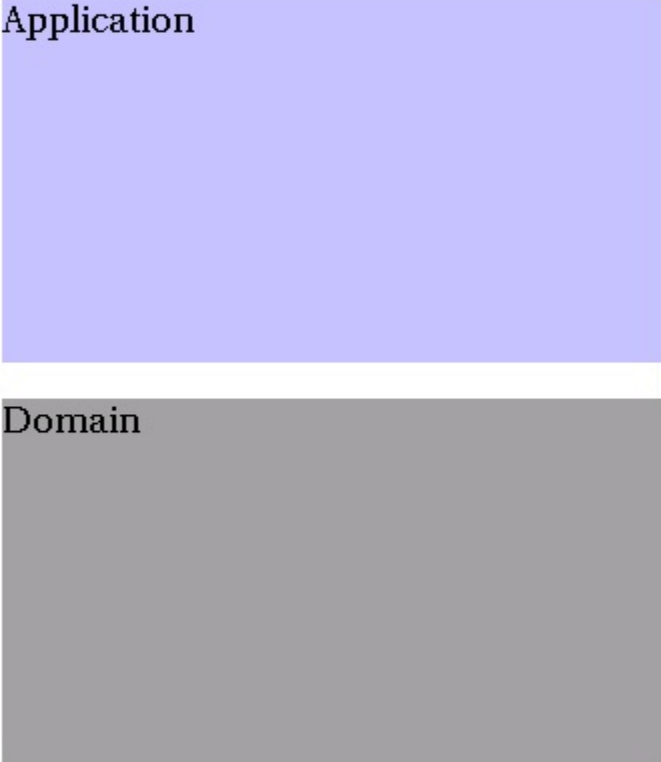
- Identify **Domain Objects** and their relations:
 - Person (Patient)
 - Address
 - HealthRecord (EHR)
 - Hospital
- Specify the **Application Components** (Main Use Cases) needed:
 - Medical Documentation (Health Record)
 - Form Printing (Report Generation)
 - Laboratory Data Retrieval
 - Billing

1.2 Analysis and Requirements



1.3 Logical Architecture (Layers)

Application



Domain

1.3 Logical Architecture - Controller

Application

ApplicationController (Process)

- every application is a tree
- the startable root is a process
- controllers can have children

Domain

1.3 Logical Architecture - View

Application

ApplicationView (Presentation)



ApplicationController (Process)

- every application is a tree
- the startable root is a process
- controllers can have children
- graphical/textual presentation

Domain



1.3 Logical Architecture - Model

Application

ApplicationView (Presentation)



ApplicationController (Process)



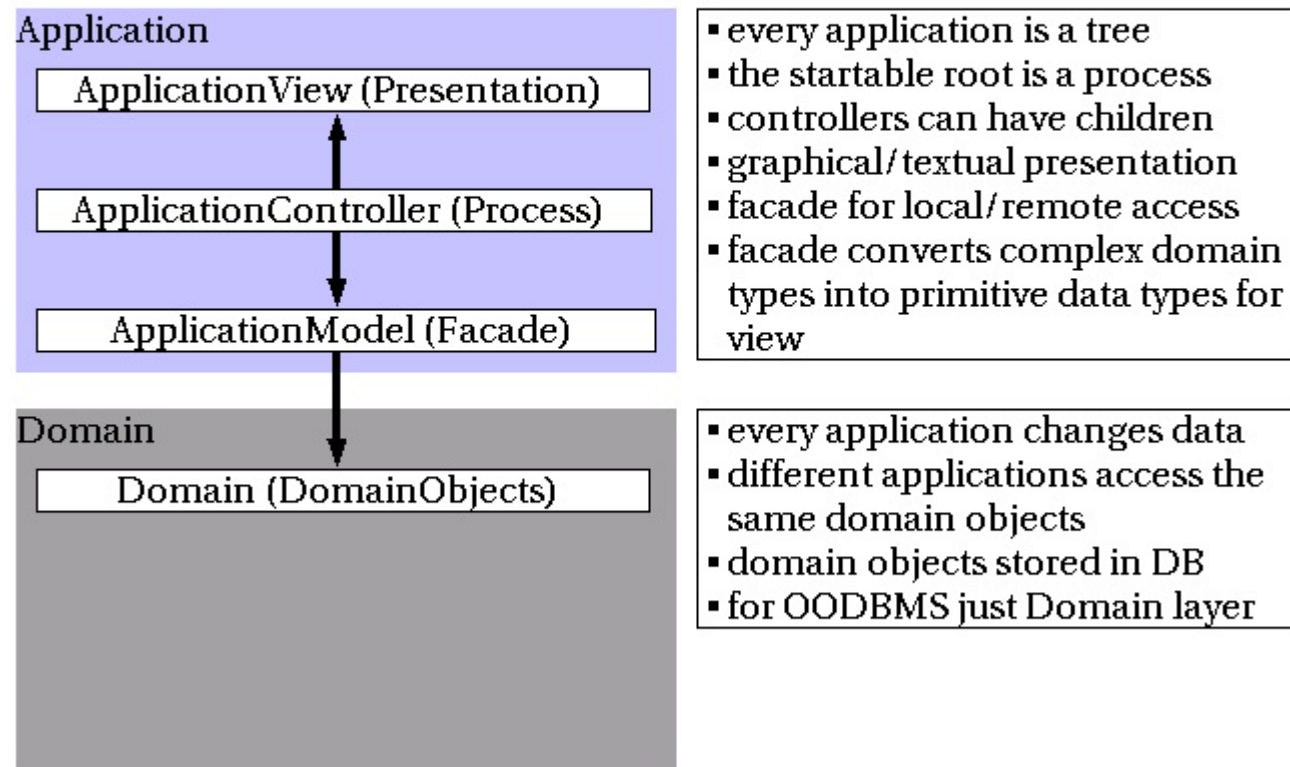
ApplicationModel (Facade)

- every application is a tree
- the startable root is a process
- controllers can have children
- graphical/textual presentation
- facade for local/remote access
- facade converts complex domain types into primitive data types for view

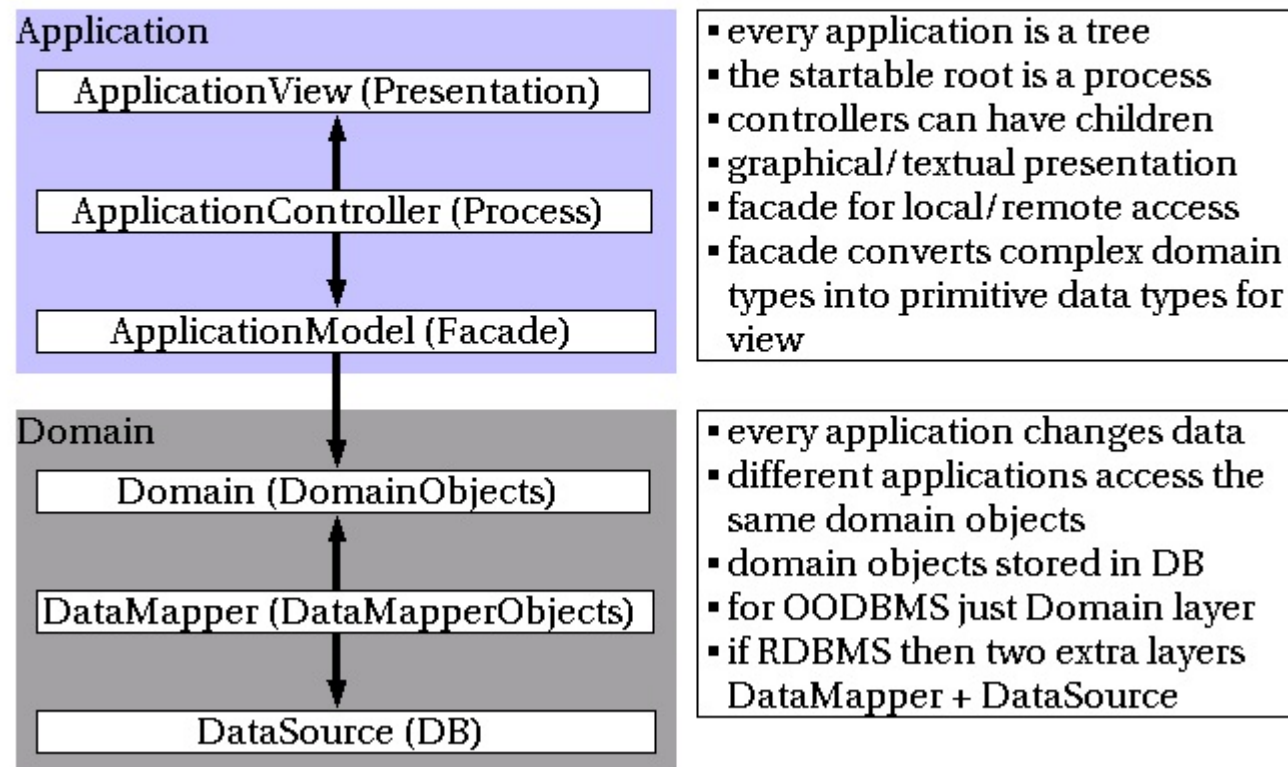
Domain



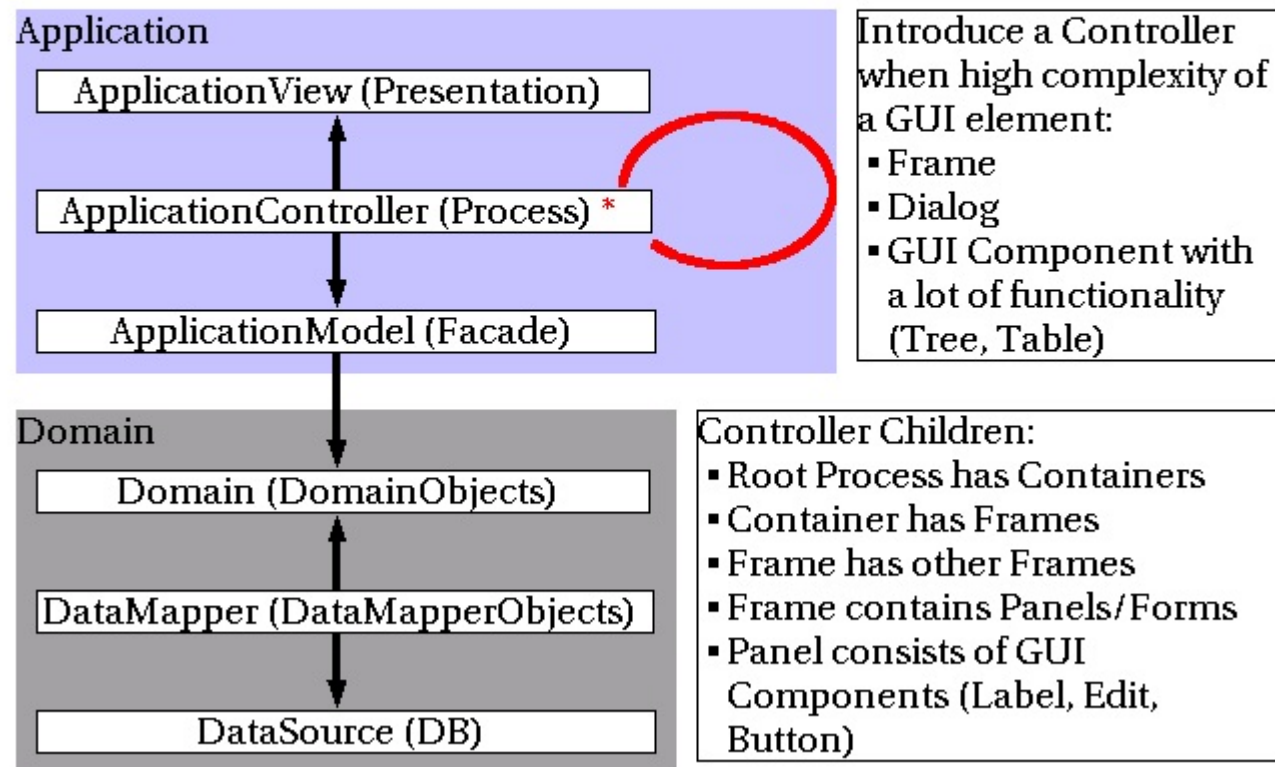
1.3 Logical Architecture - Domain



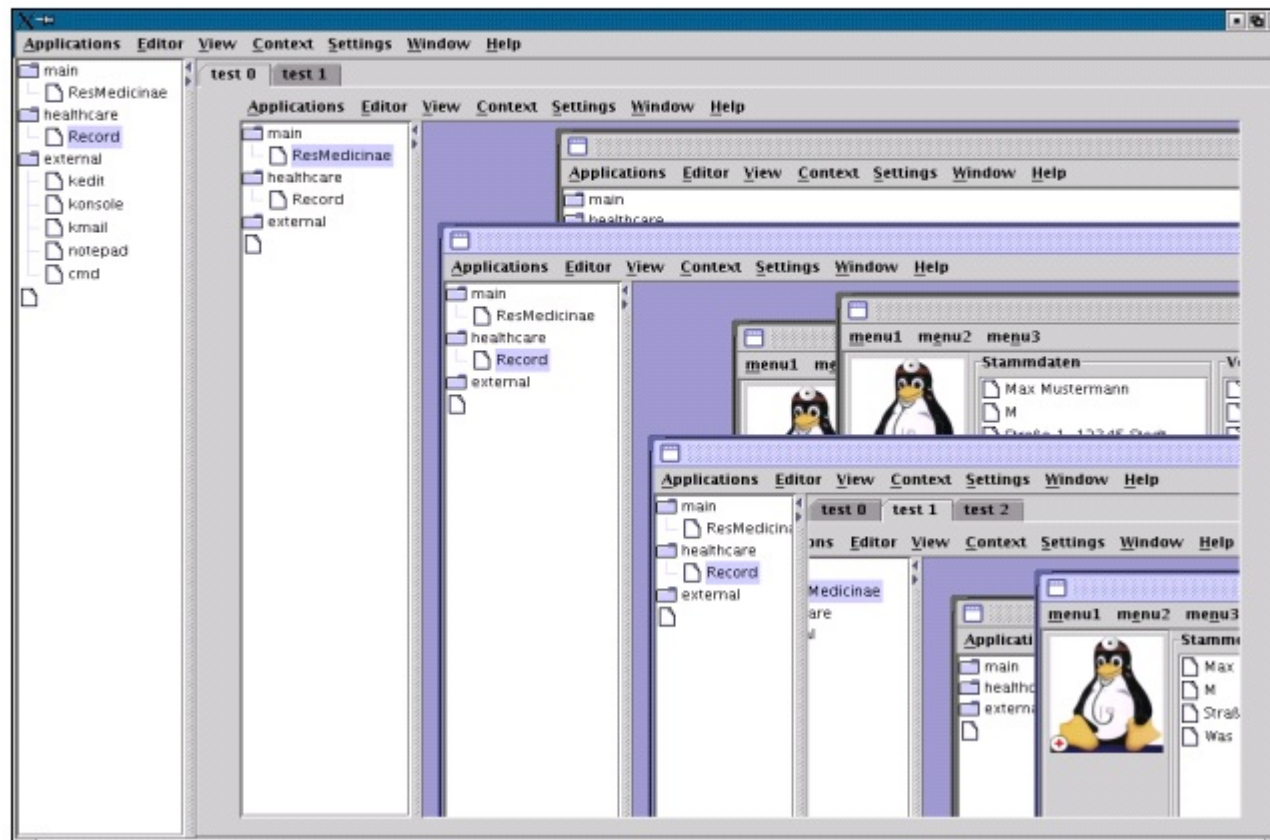
1.3 Logical Architecture - Mapper



1.3 Logical Architecture - Hierarchy



1.3 Logical Architecture - Hierarchy

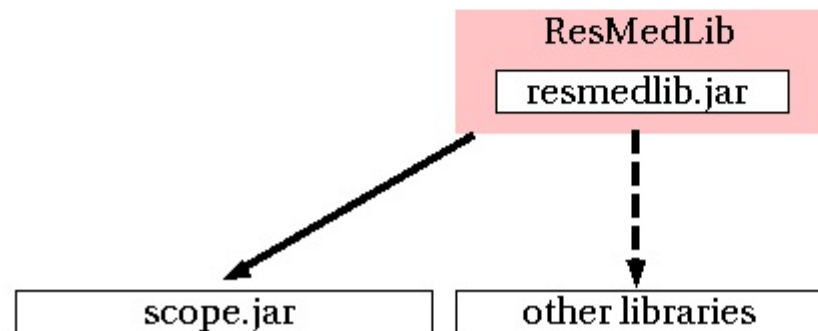


1.4 Package Dependencies

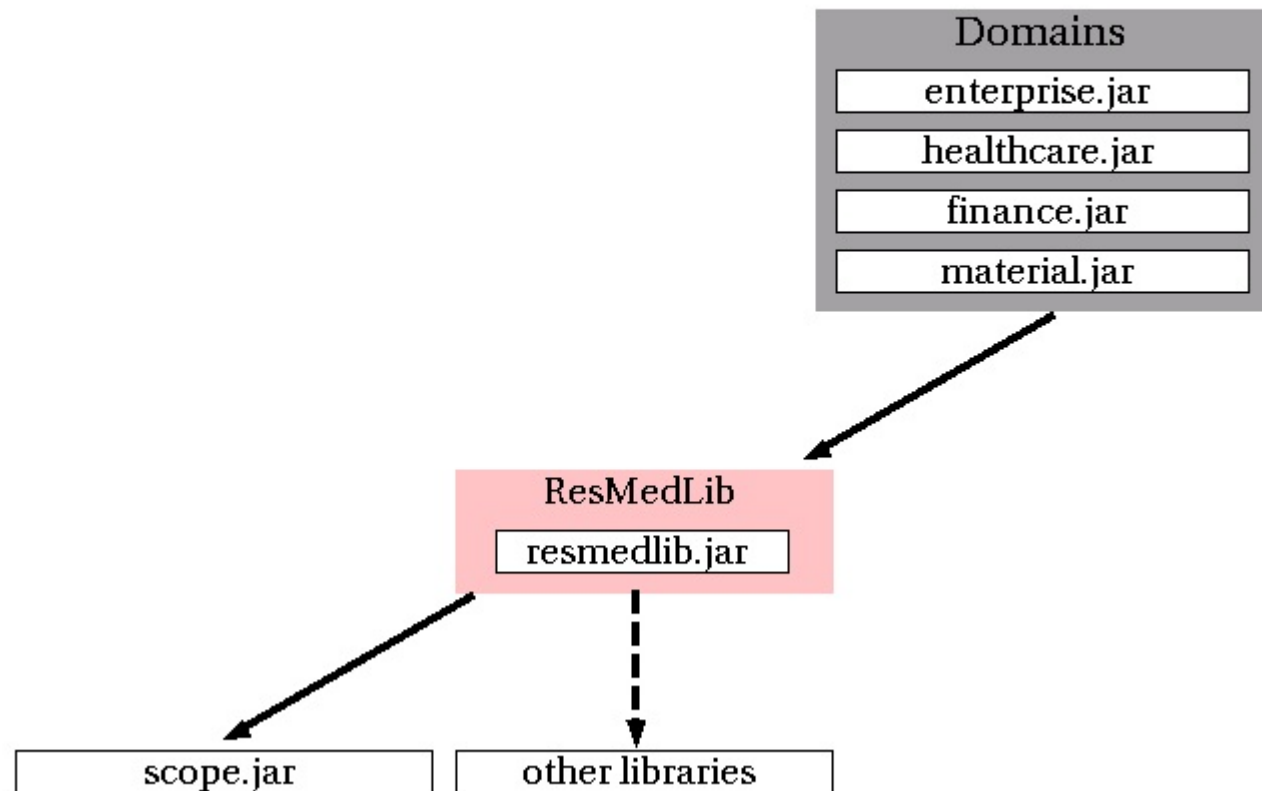
scope.jar

other libraries

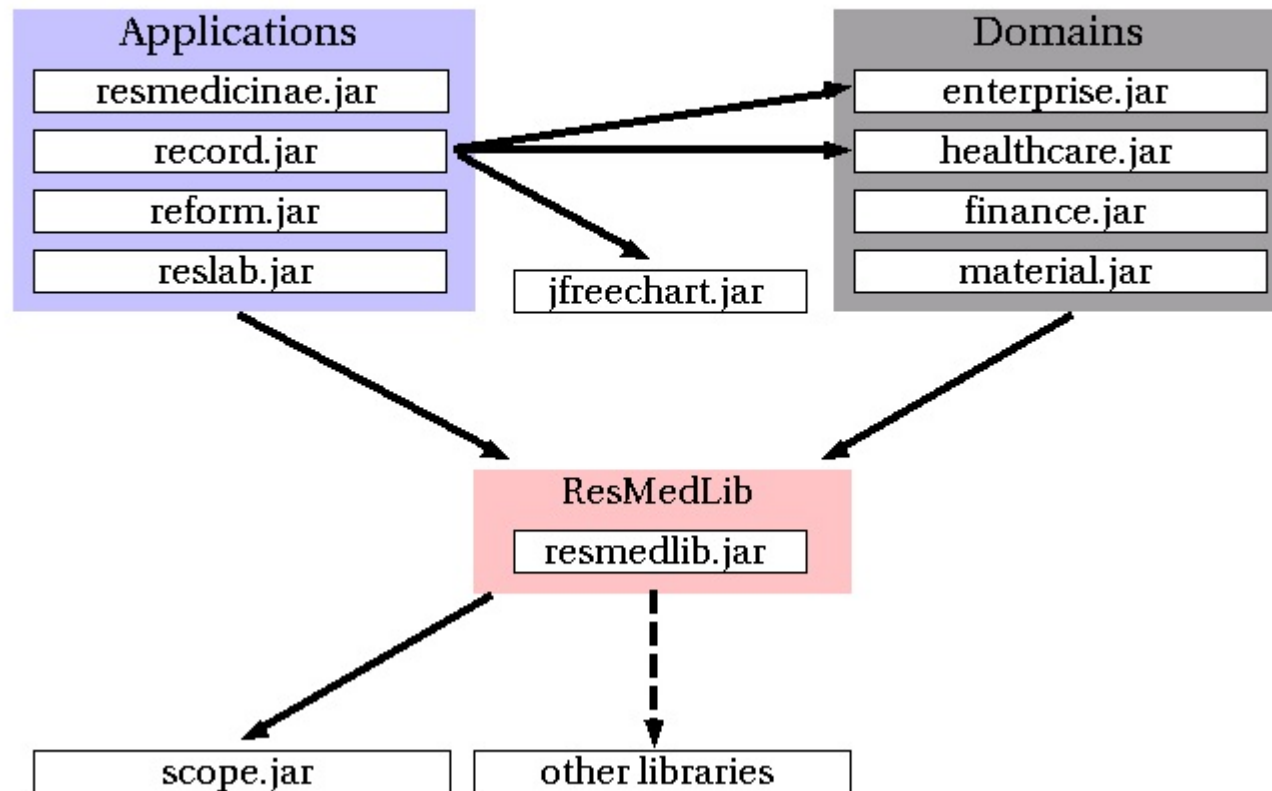
1.4 Package Dependencies



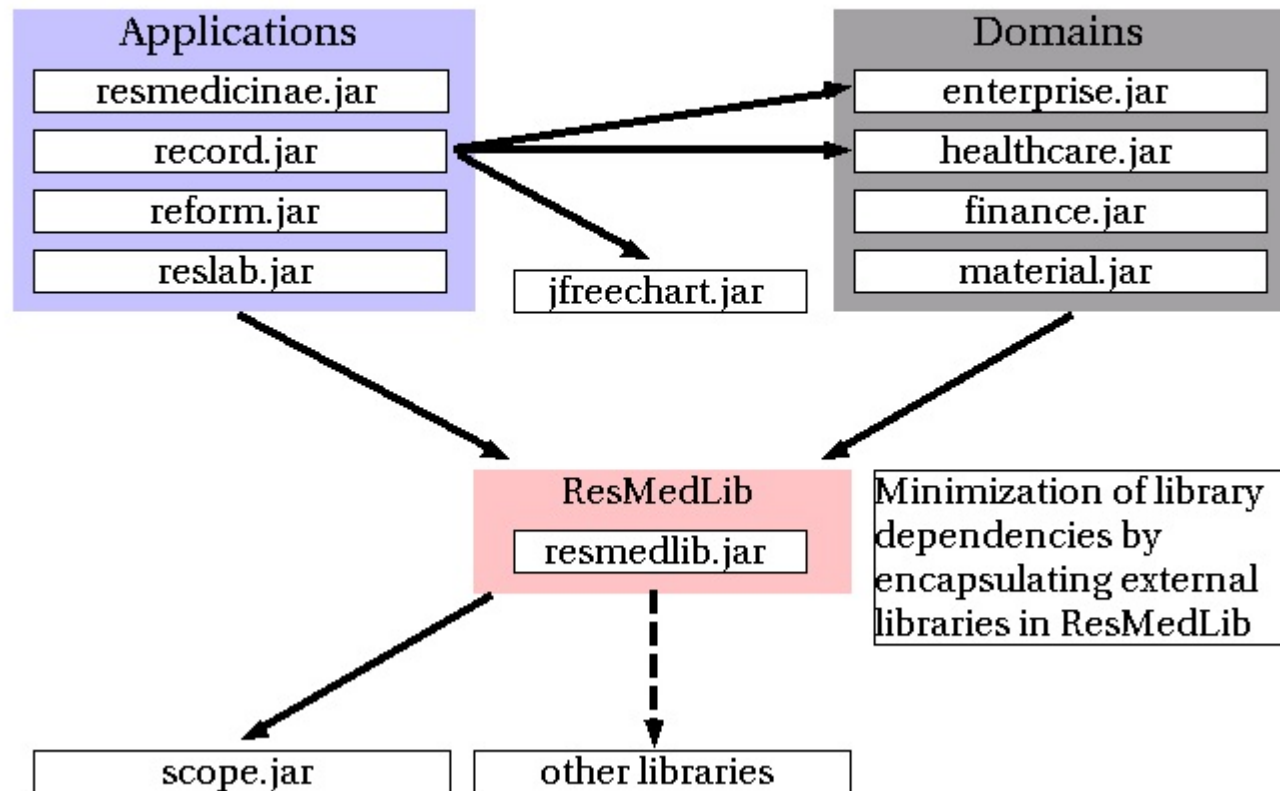
1.4 Package Dependencies



1.4 Package Dependencies



1.4 Package Dependencies



1.5 Directory Structure

/admin (project administration)

/developers

/mailinglists

/projectplan

/workgroups

/bin (build/ start/ stop scripts)

/build (compiled .class files)

/dist (distributable files)

/doc (documentation)

/etc (configuration files)

/jre (java runtime engine)

/lib (libraries, also external)

/local (things specific to the machine)

/model

/analysis (requirements, wishlist)

/design (model, structure)

/rep (UML Model/Diagrams)

/standards

/share (default system settings)

/src (source .java files)

/application

/domain

/resmedlib

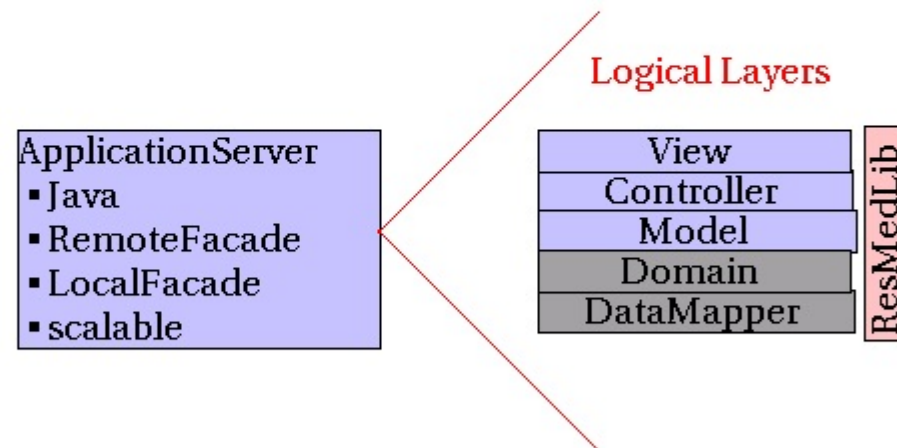
/tmp (private, temporary stuff)

1.6 Physical Architecture (Tiers)

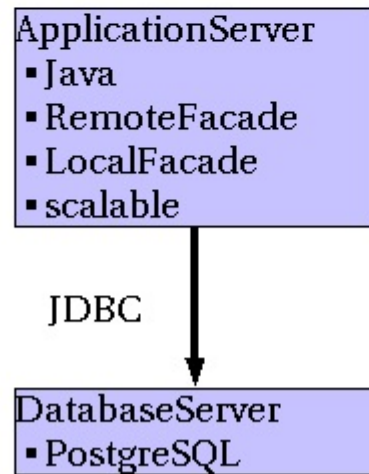
ApplicationServer

- Java
- RemoteFacade
- LocalFacade
- scalable

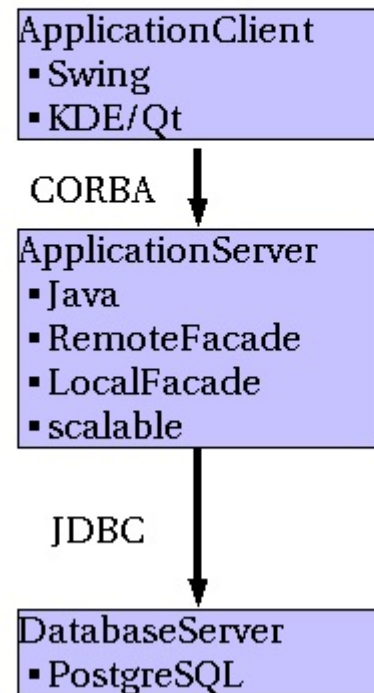
1.6 Physical Architecture - 1 Tier



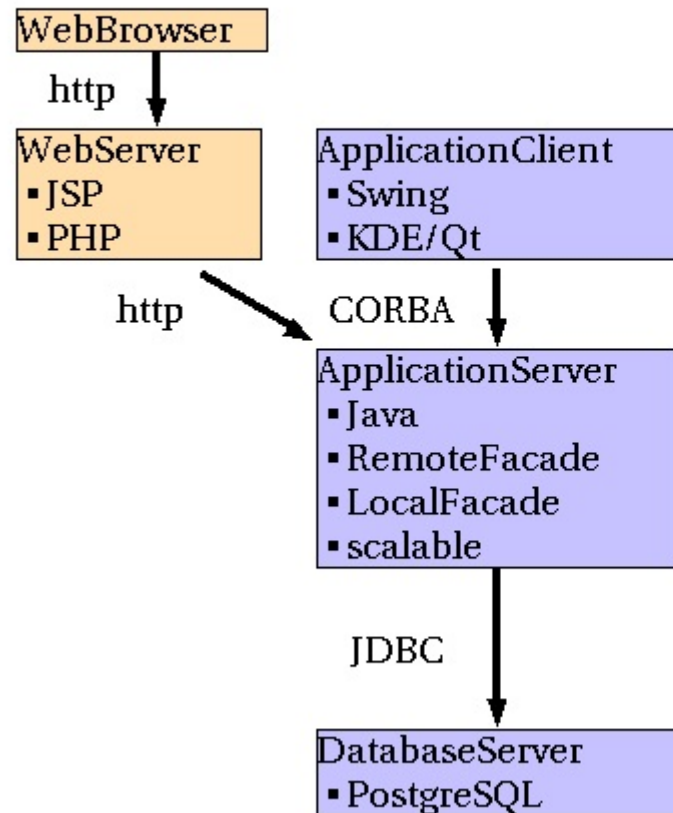
1.6 Physical Architecture - 2 Tiers



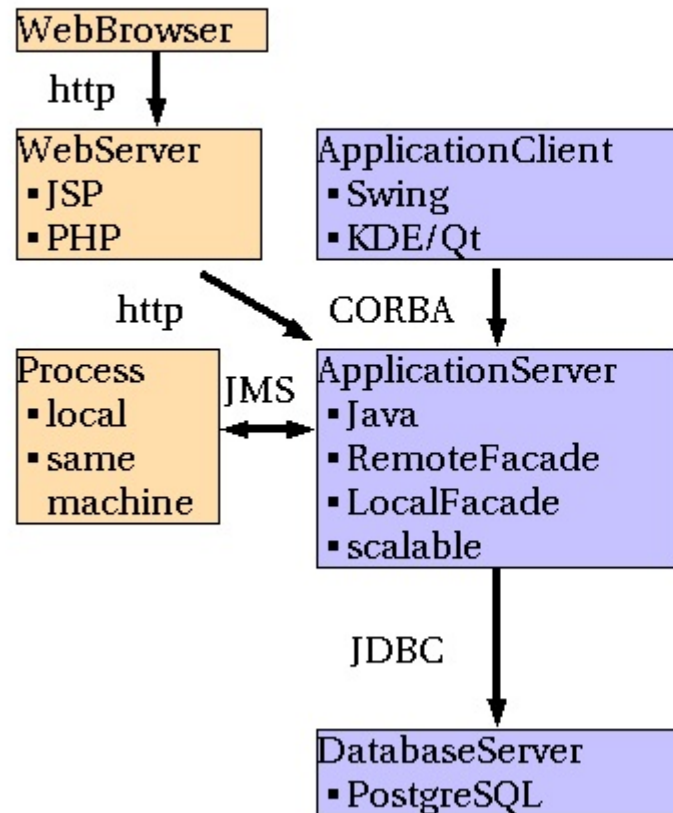
1.6 Physical Architecture - 3 Tiers



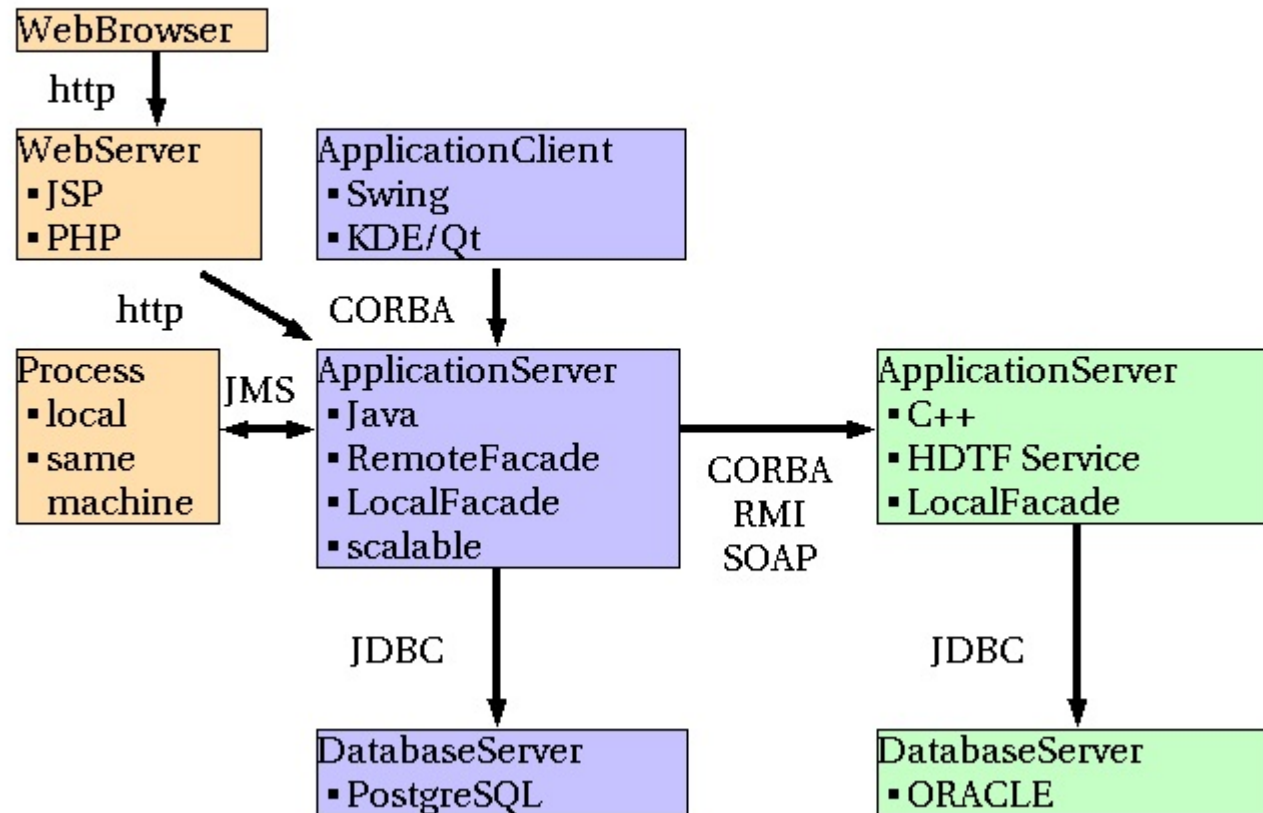
1.6 Physical Architecture - Web



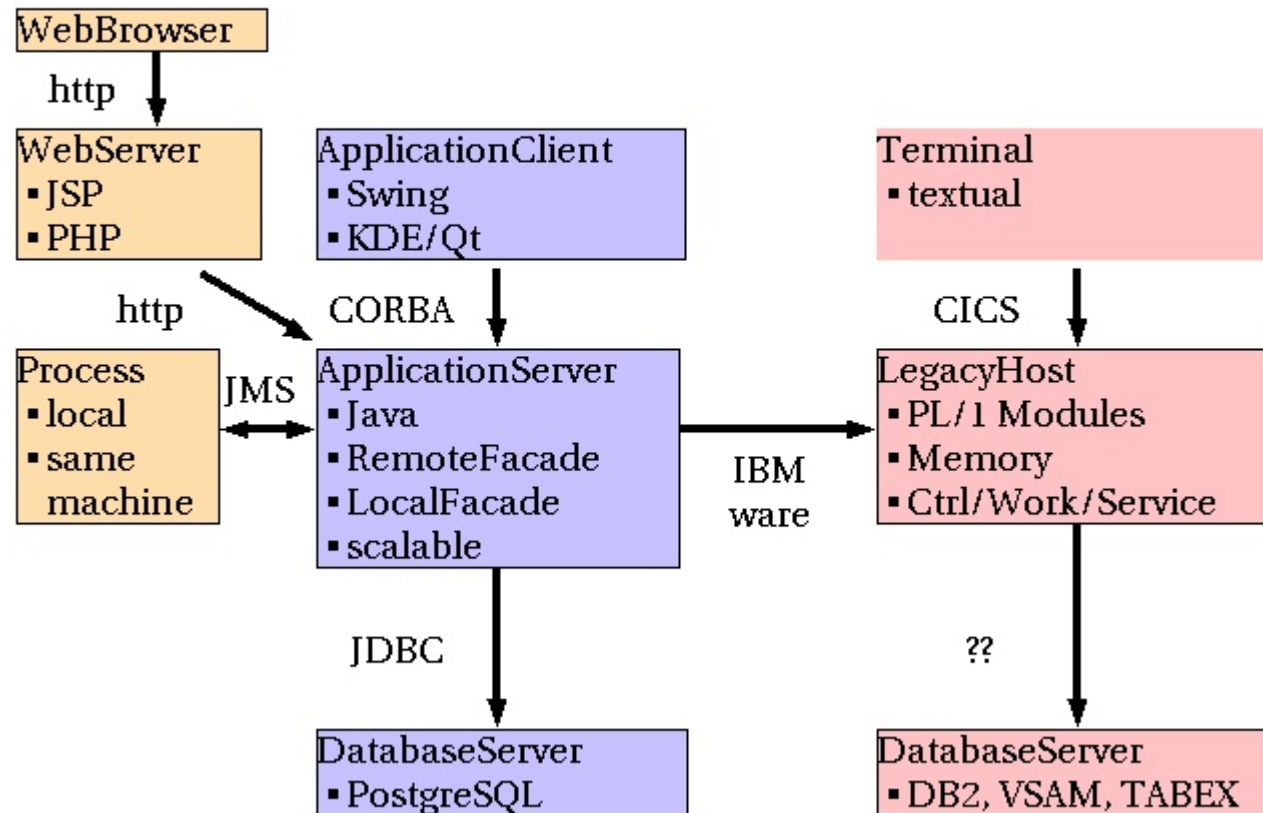
1.6 Physical Architecture - Local



1.6 Physical Architecture - Remote

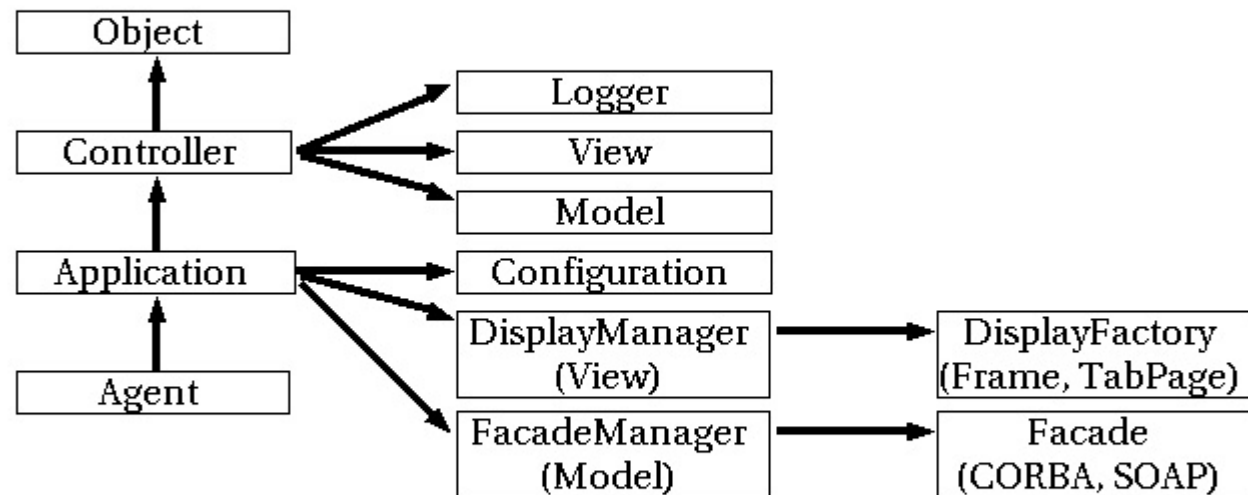


1.6 Physical Architecture - Legacy

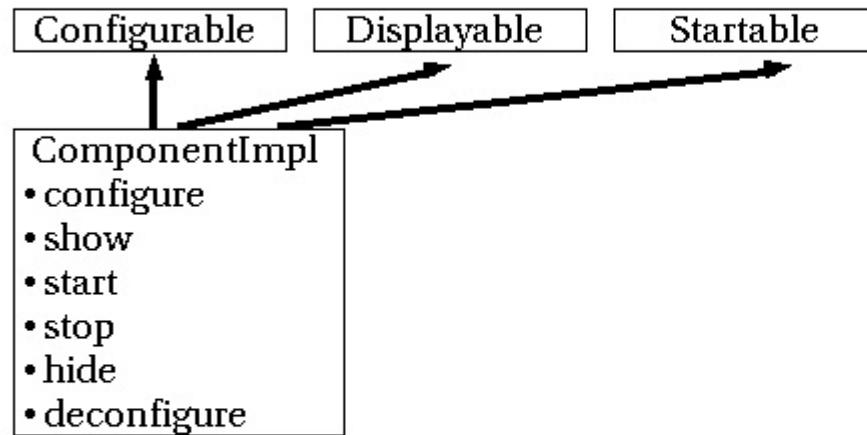


2 Application Components

2.1 Default Application



2.2 Separation of Concerns



2.3 Component Lifecycle

- 
- constructor
 - contextualize
 - compose
 - configure
 - initialize
 - prepareData
 - show
 - start
 - suspend
 - recontextualize
 - recompose
 - reconfigure
 - reparameterize
 - resume
 - stop
 - hide
 - dispose
 - finalize
 - destruct

Interfaces:

- Loggable
- Contextualizable
- Composable
- Configurable
- Initializable
- Dateable (MVC Model)
- Displayable (MVC View)
- Startable
- Suspendable
- Recontextualizable
- Recomposable
- Reconfigurable
- Reparameterizable

(c) Apache Jakarta and Res Medicinae.

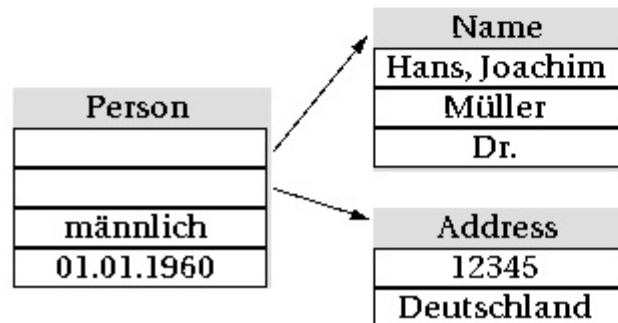
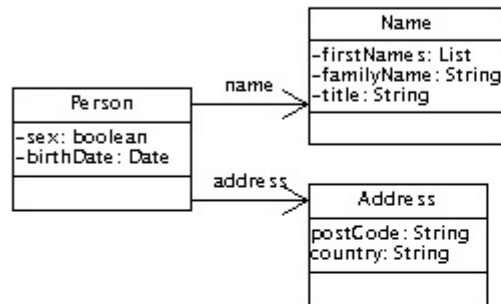
2.4 Inversion of Control

??

3 Domain Modelling

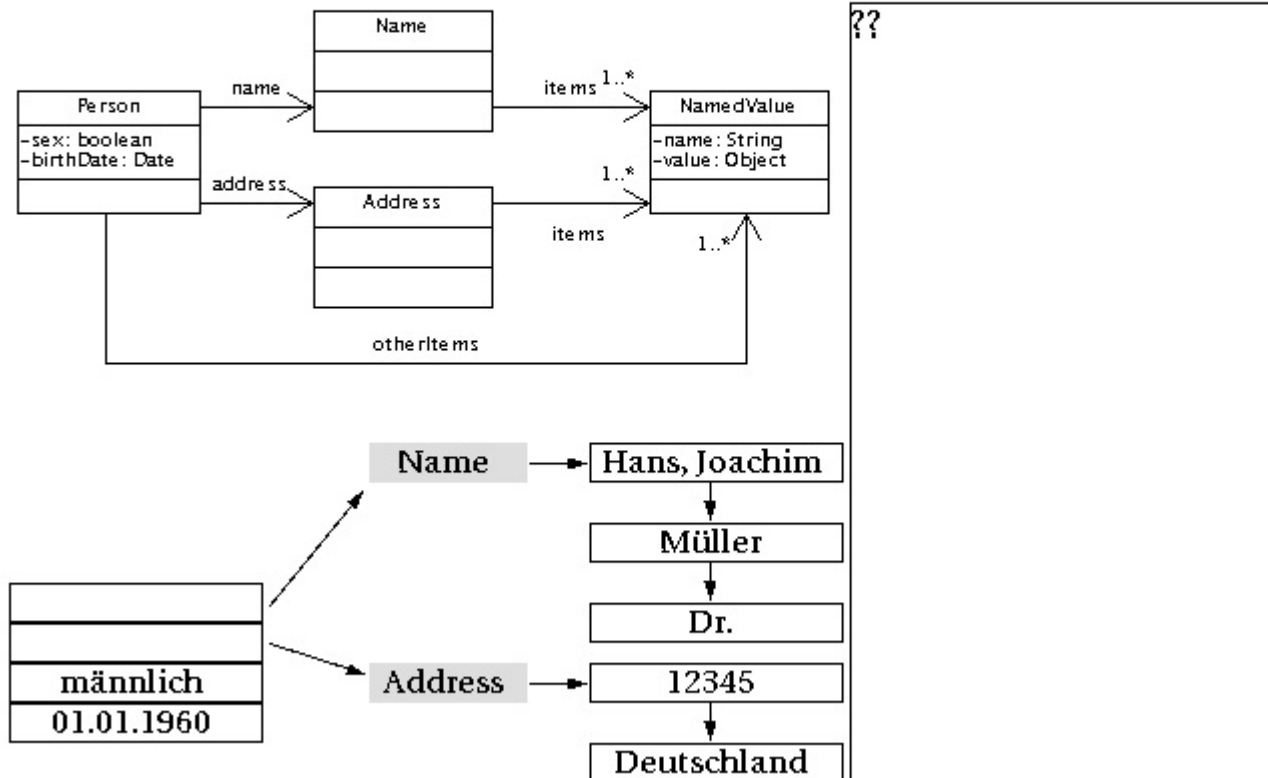
Copyright (c) 2000-2002. Thomas Beale. OpenEHR/GEHR Project.

3.1 Single Model

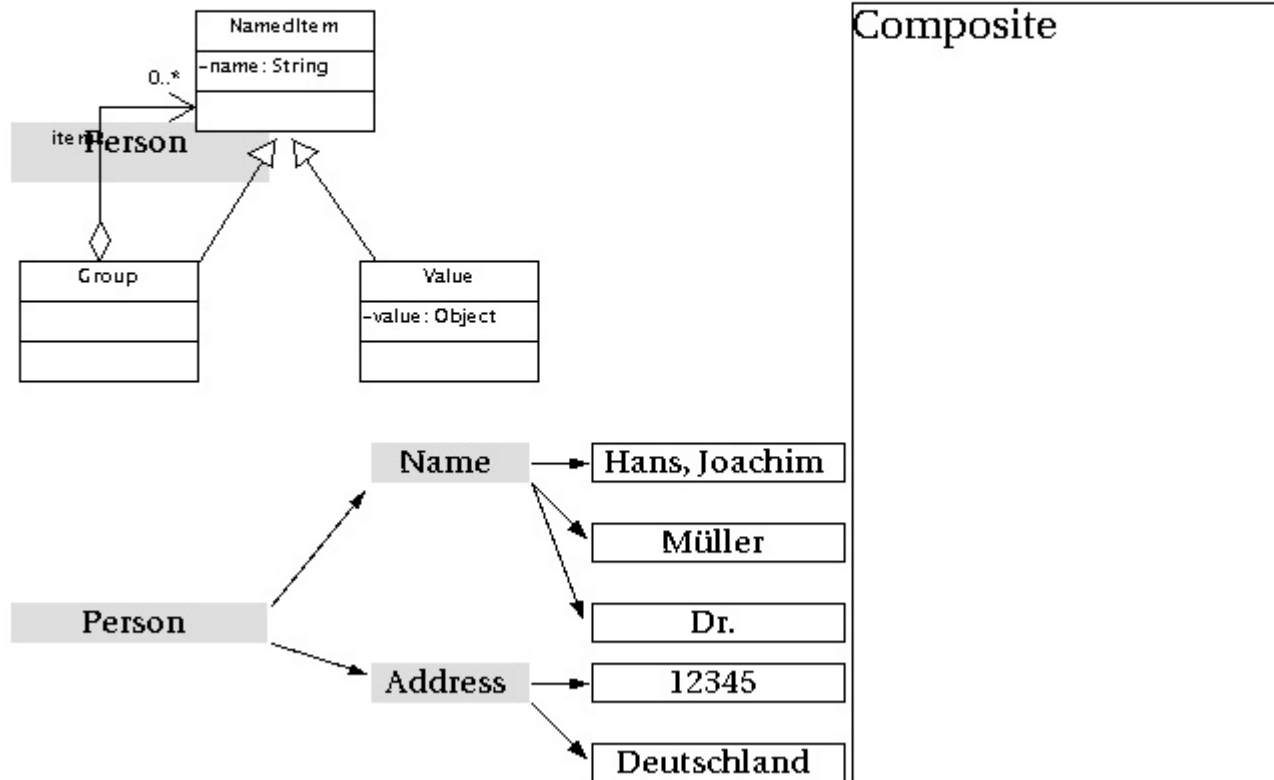


- selbst neueste OO Analyse- und Design-Methoden liefern am Ende Single-Model-System
- **SEP (iterativ):**
 - Analyse (UseCases/ Classes suchen)
 - Design (Modell erstellen)
 - Implementation (Software schreiben oder generieren)
- **Nachteile:**
 - nur Anforderungen des aktuellen Projekts berücksichtigt
 - Modelle enthalten allgemeine und semantische (Domain) Konzepte
 - Fachexperten und Software-Entwickler müssen sich abstimmen
 - für Interoperabilität ständiges Upgraden der Schnittstellen
 - Standardisierung kaum zu erreichen - schlechte Interoperabilität und automatisierte Verarbeitung (Data Mining/ Entscheidungsunterstützung)

3.2 Semi-Structured Model



3.3 Knowledge Representation



3.4 Interoperability (Dual Model)

3.5 Current Approach

- Due to the lack of a worldwide standard, all projects create their own Domain Layer using different designs
- As soon as the OpenEHR initiative produces results usable in practice, we will incorporate them into our project
- For now, we plan to set up a simple Domain Model with basic classes such as Person, HealthRecord and related entities

Thank you!

Copyright (c) 1999-2002. Christian Heller.
All rights reserved.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; with no Invariant Sections, with no Front-Cover Texts and with no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Christian Heller
Taubachstrasse 23
98714 Stuetzerbach
Germany
christian.heller@tuxtax.de
<http://www.resmedicinae.org>



Questions?

; -)

4 Appendix

4.1 Support



- Number of Developers is still small
- It took one year to figure out a clear architecture
- Now we are approaching a point where the basics are clear
- Java beginners may start creating GUI forms for modules
- Experienced developers can drop in creating new modules
- ...plenty of work besides programming (Analysis, Documentation, Translation, Webdesign)

To with-work on ResMedicinae, see the **Support/Help** section at:
<http://www.resmedicinae.org>

4.2 Abbreviations

API - Application Programmer's Interface
CICS - Customer Information Control System
CORBA - Common Object Request Broker Architecture
DB - Database
DDL - Data Definition Language
ERM - Entity Relationship Model
GUI - Graphical User Interface
HDTF - Healthcare Domain Task Force (formerly CORBAMED)
HMVC - Hierarchical Model View Controller (Design Pattern)
HTTP - Hypertext Transfer Protocol
IBM - Industrial Business Machines, Inc.
IDL - Interface Definition Language (CORBA)
JDBC - Java Database Connectivity
JMS - Java Messaging Service
JSP - Java Server Pages
KDE - K Desktop Environment

4.2 Abbreviations

OOM - Object Oriented Model

OODBMS - Object Oriented Database Management System

PL/1 - Programming Language for old Host Systems (procedural)

Qt - C++ Library of Trolltech, Base Library of KDE

RDBMS - Relational Database Management System

RMI - Remote Method Invocation (=RPC)

RPC - Remote Procedure Call (=RMI)

SOAP - Simple Object Access Protocol (XML based)

SQL - Structured Query Language

TABEX - Table Management System of BOI company

UML - Unified Modelling Language (just Diagrams)

VSAM - Virtual Storage Access Method (old hierarchical DB)

XML - Extended Markup Language

4.3 Sources

- Martin Fowler. Analysis Patterns. Muenchen: Addison-Wesley, 1997.
<http://www.martinfowler.com>
- Erich Gamma et al. Design Patterns. Muenchen: Addison-Wesley, 1996.
- GNUmed. <http://www.gnu.org>
- Hierarchical Model-View-Controller (HMVC).
http://www.javaworld.com/javaworld/jw-07-2000/jw-0721-hmvc_p.html
- Apache-Jakarta-Framework. Component-Oriented Design.
<http://jakarta.apache.org/avalon/framework/lifecycle.html>
- K Desktop Environment (KDE). Some images and icons.
<http://www.kde.org>
- Ralf Kuehnel. Agentenbasierte Softwareentwicklung. Muenchen: Addison-Wesley, 2001.
- Open Infrastructure for Outcomes (OIO). Resource Library.
<http://www.txoutcome.org/scripts/zope/library/>
- OpenEHR, GEHR. <http://www.openehr.org> <http://www.gehr.org>
- OpenEMed. <http://www.openemed.org>
- Res Medicinae. <http://www.resmedicinae.org>
- The Scope Project. <http://sourceforge.net/projects/scope>

4.4 Tools

- [Linux](http://www.linux.org) <http://www.linux.org>
- [XFree86](http://www.xfree86.org) <http://www.xfree86.org>
- [KDE](http://www.kde.org) <http://www.kde.org>
- [KPresenter](http://www.kde.org/koffice) <http://www.kde.org/koffice>
- [Gimp](http://www.gimp.org) <http://www.gimp.org>
- [ArgoUML](http://www.argouml.org) <http://www.argouml.org>
- [Kate](http://www.kde.org) <http://www.kde.org>
- [QuantaPlus](http://www.kde.org) <http://www.kde.org>
- [Konqueror](http://www.kde.org) <http://www.kde.org>

X Tutorial (planned)

X.1 Build Process

0 Run build scripts (/bin) in order:

- ResMedLib
- Domains
- Applications

That's all.

What happens?



Automatically:

- 1 Start of Java (/jre) through scripts
- 2 Compilation of .java files (/src)
- 3 Saving of .class files (/build)
- 4 Archiving and Packing .class files (/lib)

X.2 Build Process

/admin (project administration)
 /developers
 /mailinglists
 /projectplan
 /workgroups
/bin (build/ start/ stop scripts) **0**
/build (compiled .class files) **3**
/dist (distributable files)
/doc (documentation)
/etc (configuration files)
/jre (java runtime engine) **1**
/lib (libraries) **4**

/local (things specific to the machine)
/model
 /analysis (requirements, wishlist)
 /design (model, structure)
 /rep (UML Model/Diagrams)
 /standards
/share (default system settings)
/src (source .java files) **2**
 /application
 /domain
 /resmedlib
/tmp (private, temporary stuff)